

Activity 1: BFS Implementation

```
# BFS on Graph

graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = []
queue = []

def bfs(graph, start):
    visited.clear()
    queue.clear()

    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("BFS Traversal:")
bfs(graph, '5')
```

```
BFS Traversal:
5 3 7 2 4 8
```

Activity 2: BFS for a Tree Structure

```
# BFS on Tree

def printChildren(root, adj):
    queue = [root]
    visited = [False] * len(adj)

    while queue:
        node = queue.pop(0)
        visited[node] = True

        print(node, "-> ", end=" ")

        for child in adj[node]:
```

```
        if not visited[child]:
            print(child, end=" ")
            queue.append(child)
    print()

# Create tree
N = 7
adj = [[] for _ in range(N + 1)]

edges = [
    (1,2), (1,3), (1,4),
    (2,5), (2,6),
    (4,7)
]

for u,v in edges:
    adj[u].append(v)
    adj[v].append(u)

print("Children of each node:")
printChildren(1, adj)
```

[Show hidden output](#)

> Activity 3: BFS Implementation for a new graph

↳ 1 cell hidden

> Lab Task 1: BFS for a Weighted Graph

↳ 2 cells hidden